MOVIMIENTO MISIONERO MUNDIAL — VENEZUELA

Sistema SAPRCOE • Plataforma Nacional de Gestión Pastoral y Eventos

AUDITORÍA TÉCNICA • USO INTERNO

EVALUACIÓN INTEGRAL DE INGENIERÍA Y CIBERSEGURIDAD

Informe de Estado Operativo y Seguridad

Relevamiento exhaustivo módulo por módulo de la arquitectura funcional, controles de autenticación, autorización RBAC, exposición de datos personales, ecosistema móvil/PWA, microservicio de IA fotográfica y plan de endurecimiento previo a operación crítica en producción.

11 Módulos
ÁREAS FUNCIONALES ANALIZADAS

253 Rutas HTTP
WEB, ADMIN (/ADMIN) Y API V1

34 / 16 / 64
CONTROLADORES / MODELOS / MIGRACIONES

2 Apps + Studio IA
EXPO PWA (ASISTENCIA Y PRESBITERIO) + FASTAPI

45 vs 116
RUTAS ADMIN CON PERMISO CAN: VS SIN CAN:

15 Hallazgos
4 CRÍTICOS • 4 ALTOS • 4 MEDIOS • 3 BAJOS

DICTAMEN EJECUTIVO PRELIMINAR

La plataforma **SAPRCOE** presenta una madurez funcional notable para el censo pastoral nacional, emisión automatizada de credenciales con códigos QR y mejoras biométricas IA, acreditación masiva en convenciones (en línea y 100% offline mediante IndexedDB/AsyncStorage) y supervisión territorial de las 41 zonas del país. En materia de seguridad, incorpora controles modernos como Laravel Fortify con 2FA TOTP, Passkeys WebAuthn, aislamiento multi-tenant por zona/distrito (`Multitenantable`), cifrado de secretos SMTP/Mailgun/JAAK y bitácora con `spatie/laravel-activitylog`. Sin embargo, existen **brechas críticas de autorización y exposición pública** (ausencia de middleware `can:` en 106 rutas administrativas incluyendo respaldo/importación de BD y ejecución Artisan, auto-registro Fortify abierto en `/register`, terminales y API de asistencia sin autenticación ni token de dispositivo, actualización pública no verificada de fichas pastorales en `/registro`, y almacenamiento de claves en claro en la app Presbiterio) que requieren remediación prioritaria antes de habilitar operación pública masiva.

PLATAFORMA BASE
Laravel 13.40.0 / PHP 8.5.12

FRONTEND / MÓVIL
React 19 + Inertia 3 + Expo PWA

ENTORNO INSPECCIONADO
APP_ENV=local (Laragon)

FECHA DE EMISIÓN
13 de Noviembre de 2026

01 Resumen Ejecutivo y Postura Global

El análisis se efectuó mediante inspección estática completa de rutas, controladores, modelos, middlewares, proveedores de servicio, configuración, vistas React/TypeScript, aplicaciones móviles Expo/PWA y microservicio Python del repositorio `c:\laragon\www\mmmvenezuela`.

4	4	4	3
RIESGOS CRÍTICOS	RIESGOS ALTOS	RIESGOS MEDIOS	BAJOS / OPERATIVOS

Estado Operativo General

FUNCIONAL

- **Núcleo Pastoral y Territorial:** Completo de extremo a extremo (censo nacional, planillas FPDF, Excel PhpSpreadsheet, carnets frontales/traseros 85×54 mm, verificación QR e iglesias/extensiones georreferenciadas).
- **Control de Eventos Multisesión:** Operación en tiempo real y offline web/móvil sincronizada mediante bandera única `is_activa` por sesión/día, con acreditación dual automática de cónyuges y bloqueo atómico transaccional.
- **Estudio Fotográfico IA:** Microservicio `MMM Studio AI v2.0` operativo con modelos ONNX (YuNet, RMBG-1.4, MODNet, GFPGAN v1.4, Real-ESRGAN) + 14 archivos `@vladmandic/face-api` locales.
- **Brechas Funcionales Detectadas:** Las gráficas en vivo de *Monitoreo de Servidor* y *Métricas de Base de Datos* devuelven telemetría simulada (`rand()`); existen comandos/permisos heredados de módulos ajenos (nómina LFT, productores/proveedores) sin UI activa.

Postura de Seguridad General

ACCIÓN REQUERIDA

- **Controles Sólidos Presentes:** Autenticación Laravel Fortify con 2FA TOTP confirmado, Passkeys WebAuthn, limitación de tasa en login/2FA/passkeys, rotación inicial obligatoria de contraseña, aislamiento multi-zona (`Multitenantable`), auditoría `spatie/laravel-activitylog` y re-verificación de contraseña para backup BD y documentos.
- **Superficie Crítica Expuesta:** La política RBAC definida en `PermissionSeeder` no se aplica en el enrutamiento de 106 endpoints `/admin/*` ; `/register` está abierto; los terminales y la API de asistencia operan sin credenciales; `/registro` permite alterar fichas existentes y subir archivos sin validar MIME; y la PWA Presbiterio guarda contraseñas en claro en `AsyncStorage` .

Mapa Consolidado de Riesgos Verificados

ID	SEVERIDAD	HALLAZGO / BRECHA DE SEGURIDAD	COMPONENTE AFECTADO	IMPACTO PRINCIPAL
C1	CRÍTICO	Ausencia de middleware `can:` en 106 rutas `/admin/*` y ejecución Artisan sin lista blanca	`routes/modules/*.php` , `DbMonitoringController` , `TaskMonitoringController`	Escalamiento de privilegios, volcado/restauración BD, comandos Artisan
C2	CRÍTICO	Auto-registro público Fortify habilitado sin asignación de tenant ni restricción de rol	`config/fortify.php` , `CreateNewUser.php` , `/register`	Creación externa de usuarios autenticados en combinación con C1
C3	CRÍTICO	Endpoints del terminal web y API v1 de asistencia públicos sin autenticación + seed estático	`routes/web.php` , `routes/api.php` , `AsistenciaController` , `padron-seed.json`	Fuga del padrón nacional (899+ pastores), alteración/borrado de asistencias
C4	CRÍTICO	Exposición masiva PII/salud, sobreescritura de fichas existentes y subida sin validación MIME	`PastorRegistroPublicoController` , `enrolarRostro` , `pastores_cedulas`	Secuestro/alteración de fichas, fuga médica y subida de archivos peligrosos
A1	ALTO	Microservicio FastAPI IA en `0.0.0.0:8001` sin autenticación, CORS abierto y rutas de disco libres	`app/Scripts/ai_server.py` , `ai_engine.py`	Lectura/escritura arbitraria de imágenes en disco vía API abierta

ID	SEVERIDAD	HALLAZGO / BRECHA DE SEGURIDAD	COMPONENTE AFECTADO	IMPACTO PRINCIPAL
A2	**ALTO**	Contraseñas en texto plano en `AsyncStorage` y WebAuthn puramente local en cliente	`presbiterio-app` (`AuthContext.tsx` , `secureStore.ts` , `biometricService.ts`)	Robo de credenciales en navegador/dispositivo vía XSS o acceso físico
A3	**ALTO**	Webhook WhatsApp exento de CSRF sin validación HMAC/secreto compartido	`bootstrap/app.php` , `WhatsAppWebhookController.php`	Bajas forzadas (opt-out) de usuarios e inyección de mensajes/ACK falsos
A4	**ALTO**	Flujo OTP sin throttle ni bloqueo de intentos, `mt_rand()` , OTP loggeado en claro y clave `min:8`	`ForgotPasswordOtpController.php` , `routes/web.php`	Fuerza bruta sobre OTP de 6 dígitos y exposición de códigos en logs
M1	**MEDIO**	Secretos en props Inertia, campos sin ocultar/cifrar en `Empresa` y hash de clave en `activity_log`	`IntegrationController.php` , `Empresa.php` , `User.php`	Exposición de llaves SMTP/Mailgun/ControlAcceso en DOM y registros
M2	**MEDIO**	Envío de claves temporales en claro por WhatsApp/Email CC, fallback texto plano y clave de prueba	`UserController.php` , `FortifyServiceProvider.php` , `WhatsAppService.php`	Exposición de credenciales iniciales en buzones compartidos y legado
M3	**MEDIO**	Contradicción política `max:12` vs `min:12` producción y tokens Sanctum sin expiración	`ForceChangePasswordController.php` , `AppServiceProvider.php` , `sanctum.php`	Bloqueo o menor entropía en contraseñas y tokens móviles perpetuos
M4	**MEDIO**	Sin cabeceras HTTP de seguridad (CSP/HSTS/XFO), CORS `*` y `APP_KEY` pre-poblada en ejemplo	`app/Http/Middleware` , `public/.htaccess` , `cors.php` , `.env.example`	Menor defensa contra clickjacking, MIME sniffing e incidentes XSS
B1	**BAJO**	Enumeración secuencial por ID/cédula en verificación pública de carnet QR sin throttle	`PastorCarnetController::validarCredencial` , `routes/web.php`	Recolección automatizada de identidad y estatus ministerial
B2	**BAJO**	Telemetría en vivo simulada con `rand()` y mención a tablas ajenas (`leads` , `larareact`)	`ServerMonitoringController.php` , `DbMonitoringController.php`	Información engañosa durante incidentes operativos de CPU/RAM/BD
B3	**BAJO**	Typo `'inactivos'` en conteo, permisos/comandos LFT residuales y debug local activo	`UserController::index` , `PermissionSeeder` , entorno actual	KPI erróneo de usuarios inactivos y deuda de mantenimiento

02 Arquitectura Tecnológica y Ecosistema

SAPRCOE opera como una plataforma monolítica modular en Laravel 13 complementada por clientes React/Inertia, dos aplicaciones móviles/PWA en Expo y un microservicio especializado de visión artificial en Python.

CAPA DE PRESENTACIÓN Y CLIENTES OFFLINE	**Panel Web Admin & Público** React 19.2 + TypeScript + Inertia.js 3.0 + Tailwind/Shadcn + Vite 8 PWA (86 páginas `.tsx`)	**Terminal Escáner Offline Web** `Escanear.tsx` + `offlineAsistenciaWeb.ts` (IndexedDB) + `jsqr` + `@vladmandic/face-api`	**Apps Móviles & PWA Expo** `asistencia-app` (Padrón seed 899 + cámara) y `presbiterio-app` (`/presbitero` , Push & Biometría)
SEGURIDAD, ENRUTAMIENTO Y GATEWAYS	**Autenticación Web** Laravel Fortify 1.37 (Login dual, TOTP 2FA, `@laravel/passkeys` , OTP WhatsApp, Cambio Forzado)	**API & Multi-Tenancy** Sanctum 4.3 (`auth:sanctum`), Spatie Permission 8.3, Trait `Multitenantable` (Empresa/Sucursal/Zona)	**Middlewares & Proxy** `trustProxies('*')` , `RegionalConfiguration` , `SetLocale` , `EnsurePublicRegistrationIsActive`
LÓGICA DE NEGOCIO Y PROCESAMIENTO	**Dominio & Servicios (20)** 34 Controladores, 16 Modelos, `CarnetService` , `PlanillaService` , Exports Excel/FPDF/DomPDF	**Microservicio IA (Puerto 8001)** Python FastAPI (`ai_server.py`): YuNet, RMBG-1.4, MODNet, GFPGAN v1.4 y Real-ESRGAN ONNX	**Tareas Programadas y Colas** Queue driver `database` , Scheduler cada 1m (`asistencia:verificar-descansos`) y 5m (`whatsapp:heartbeat`)
DATOS E INTEGRACIONES EXTERNAS	**Persistencia & Auditoría** MySQL (64 migraciones), Sesiones en BD, Caché en BD, `spatie/laravel-activitylog` , `public/pastores` (1.900 archivos)	**Mensajería & Correo** WhatsApp Gateway API (Spintax, Blacklist, Webhooks), Google SMTP, Mailgun, Mailpit, Expo Push API	**Identidad, Mapas & IVMS** JAAK KYC (Sandbox/Prod), Control de Acceso IVMS, Mapbox GL y Google Maps / Leaflet

Autenticación y MFA
Fortify 1.37, 2FA TOTP confirmado, Passkeys WebAuthn, Sanctum 4.3, OTP WhatsApp.

Motor Documental
Carnets FPDF (85×54 mm) + QR BaconQrCode, Planillas FPDF, Reportes DomPDF y PhpSpreadsheet.

Visión & Biometría IA
FastAPI ONNX (YuNet, RMBG-1.4, MODNet, GFPGAN v1.4, Real-ESRGAN) + `face-api` + `MediaPipe` .

Cartografía y Push
Leaflet + Mapbox GL + Google Maps para sedes/extensiones; Expo Push API + WhatsApp Gateway.

Inventario Verificado del Código Fuente

ARTEFACTO / MÉTRICA	CANTIDAD / VERSIÓN	EVIDENCIA EN REPOSITORIO
Framework Backend / PHP	Laravel `13.40.0` / PHP `8.5.12`	`composer.json` y kernel Artisan
Rutas HTTP Totales Registradas	**253 rutas**	45 `can:`, 106 `auth+verified`, 10 admin sin auth, 14 API, 78 Web
Archivos de Rutas Modulares	**20 archivos**	`routes/modules/*.php`
Controladores / Servicios	**34** controladores / **20** servicios	`app/Http/Controllers`, `app/Services`
Modelos / Migraciones BD	**16** modelos / **64** migraciones	`app/Models/*.php`, `database/migrations/*.php`
Vistas Frontend TSX	**86 archivos** (`.tsx`)	`resources/js/pages/`
Comandos de Consola Artisan	**76 comandos**	`app/Console/Commands/*.php`
Padrón Semilla Móvil / Modelos IA	**899** pastores / **14** archivos	`padron-seed.json` (520 KB), `public/models`

Modelo de Roles, Zonas y Aislamiento (`Multitenantable`)

El trait `App\Traits\Multitenantable` inyecta un *Global Scope* (`multitenancy`) y autocompletado al crear registros sobre los modelos que lo implementan (incluidos `User`, `Pastor`, `Iglesia`, `Actividad`, `Empresa`):

- **Visibilidad Nacional:** `Super Administrador`, `super-admin`, `Supervisor Nacional`, `Junta Nacional` y `Secretaria Nacional` consultan las 41 zonas del país (manteniendo filtro por `empresa_id` / `sucursal_id` salvo Super Admin).
- **Visibilidad Regional (`Presbitero`):** Filtra automáticamente por `zona` / `zona_2` (normalizando formatos `"9"`, `"09"`, `"Zona 9"`) o por `distrito` / `distrito_2`.
- **Excepción Explícita:** La tabla `actividades` omite el filtro por zona para permitir participación nacional en convenciones; el terminal offline y `withoutTenant()` omiten el scope para búsquedas nacionales en puerta.
- **Roles en `RoleSeeder` (6):** `Super Administrador`, `super-admin`, `Supervisor Nacional`, `Presbitero`, `Junta Nacional`, `Secretaria Nacional`.

Observación Operativa sobre Deuda Técnica de Plantilla Base: Tanto `PermissionSeeder` (permisos de `empleados.*`, `proveedores.*`, `productores.*`, `visitas_temporales.*`, `asistencia.nomina` LFT) como `app/Console/Commands/` (76 comandos incluyendo recordatorios de citas, tasas de cambio y comprobantes LFT) conservan código heredado de una plantilla multi-vertical previa, aunque el sistema en operación se enfoca exclusivamente en el ecosistema pastoral MMM Venezuela.

03 Análisis Funcional y Seguridad por Módulos

Resumen integral de los 11 módulos funcionales que conforman el sistema SAPRCOE, detallando su alcance operativo real y sus controles de seguridad específicos.

Módulo 1 — Panel Principal Nacional (Dashboard Analítico)

OPERATIVO REAL | CAN:DASHBOARD.VIEW

Ruta y Controlador: `GET /dashboard` → `DashboardController@index` (middleware `auth`, `verified`, `can:dashboard.view`).

Alcance Operativo: Presenta indicadores del padrón pastoral en tiempo real: totales activos/inactivos, distribución por grado ministerial (`Pastor Asociado/Colaborador`, `Laico`, `Licenciado`, `Ministro Ordenado`), bautizados en el Espíritu Santo, cargos nacionales, estado civil, salud general, rangos de edad (<30, 30-45, 46-60, 60+), formación teológica, Top 8 estados, distribución gráfica por Zona (ApexCharts) y cumpleañeros del mes.

Seguridad y Control: Única vista general con permiso explícito en `routes/web.php`. Aplica automáticamente el scope `Multitenantable` por empresa/zona/distrito según el rol conectado. No expone contraseñas ni secretos externos.

Módulo 2 — Censo y Gestión Pastoral (`Pastores`)

OPERATIVO REAL | SIN CAN: EN RUTAS

Rutas y Controladores: `routes/modules/pastores.php` → `PastorController`, `PastorPlanillaController`, `PastorExportController`.

Alcance Operativo: Gestión integral de la ficha pastoral en wizard multipaso: datos personales/cédula, grado ministerial, promoción, vida espiritual, formación académica/teológica, domicilio completo, estado de salud (grupo sanguíneo, enfermedades crónicas, medicamentos, alergias, contacto de emergencia) y vinculación bidireccional automática de cónyuges (`conyuge_id`). Genera planilla PDF ministerial (`PlanillaService` + FPDF con auto-recorte facial 3:4), exportación Excel (`PastorExcelExportService`), descarga de copia de cédula (`descargarCedula`), activación/inactivación y eliminación masiva (`bulkDestroy`).

Seguridad y Brechas: Las consultas usan `Multitenantable` por zona/distrito. Sin embargo, las rutas bajo `/admin/pastores/*` solo exigen `auth + verified` sin middleware `can:pastores.*` ni verificación en controlador; por ello un usuario con rol `Presbitero` (que en `RoleSeeder` solo tiene `pastores.view/edit`) podría invocar directamente `POST /bulk-destroy` o crear/eliminar registros vía endpoints directos.

Módulo 3 — Carnets Digitales, Estudio Fotográfico IA y Verificación QR

OPERATIVO REAL
STUDIO IA ABIERTO + QR ENUMERABLE

Componentes: `PastorCarnetController`, `CarnetService`, `PhotoAiEnhancerService`, `app/Scripts/ai_server.py`, `designer_agent.py`.

Alcance Operativo: Centro interactivo de carnetización (`/admin/pastores/carnets`) para generar credenciales PDF frontales/traseros a escala oficial (`85 × 54 mm`, FPDF + código QR `BaconQrCode`) de forma individual o en lote (`bulkCarnetPdf`), personalizar plantillas frente/atrás y gestionar el servidor de fotos con fallback tolerante (`/pastores/{filename}`). Integra el agente **MMM Studio AI v2.0** en FastAPI (puerto 8001) con autoinicio local o remoto (`PHOTO_STUDIO_API_URL`) para analizar calidad, remover fondos (`RMBG-1.4` / `MODNet`), restaurar rostros (`GFPGAN v1.4`) y mejorar lotes (`bulkIaEnhance`).

Seguridad y Brechas: `PastorCarnetController` sí implementa control estricto interno (`autorizarAccesoCarnets()`) limitando el acceso a `Super Administrador`, `Administrador`, `Secretaria Nacional` y `Junta Nacional`. No obstante: (1) el microservicio Python `ai_server.py` en `0.0.0.0:8001` carece de token/API Key, acepta CORS `*` con credenciales y permite lectura/escritura por ruta arbitraria en disco (**A1**); y (2) la verificación pública QR `/validar-credencial/{codigo}` busca también por `id` y `documento` sin rate limiting (**B1**).

Módulo 4 — Registro Público de Pastores y Extensiones (Autogestión)

CONTROLADO POR SWITCH
FUGA PII / TOMA DE FICHA / UPLOAD

Componentes: `PastorRegistroPublicoController`, `EnsurePublicRegistrationIsActive`, `RegistroPublicoMonitoringController`.

Alcance Operativo: Portal público de 6 pasos en `/registro` (activable/desactivable desde `/admin/monitoring/registration` con permiso `monitoreo.view`). Permite verificar existencia por cédula, actualizar o crear ficha ministerial completa, capturar foto de perfil/cédula, crear municipios o parroquias al vuelo y registrar su iglesia/extensión en `/registro/{pastor}/extension` sincronizando cónyuges en la tabla pivote `iglesia_pastor`.

Seguridad y Brechas: Cuando el interruptor está abierto, el riesgo es **crítico (C4)**: `GET /registro` entrega la lista de todos los pastores activos en Inertia; `GET /registro/verificar-cedula/{cedula}` expone datos personales, domicilio e historial médico sin autenticación ni OTP; `POST /registro` permite modificar cualquier ficha existente pasando su `pastor_id` o cédula; y `foto` / `foto_cedula` no validan `image` / `mimes` al subirse al disco `public`.

Módulo 5 — Iglesias y Extensiones Pastorales

OPERATIVO REAL
SIN CAN: EN RUTAS

Componentes: `ExtensionController`, `routes/modules/extensiones.php`, modelo `Iglesia`.

Alcance Operativo: Panel estadístico congregacional (tendencias temporales ApexCharts `7d/1m/3m/1y/all`, membresía activa y probante, campos blancos, iglesias fundadas y medios de comunicación), mapa georreferenciado nacional, CRUD completo de sedes/extensiones, vinculación múltiple de pastores/cónyuges y bóveda documental por extensión.

Seguridad y Brechas: Implementa re-verificación de contraseña (`verifySecurity`) para acciones sensibles sobre documentos y aplica scope territorial `Multitenantable`. Sin embargo, todas las rutas (incluyendo `DELETE` y `bulk-destroy`) carecen de middleware `can:extensiones.*`, permitiendo por ejemplo a un `Presbitero` invocar borrado masivo vía petición HTTP directa.

Módulo 6 — Gestión de Actividades y Terminales Web de Asistencia / Biometría / Offline

OPERATIVO REAL
TERMINAL WEB SIN AUTH

Componentes: `ActividadController`, `AsistenciaController`, `AsistenciaExcelExportService`, `Escanear.tsx`, `offlineAsistenciaWeb.ts`.

Alcance Operativo: Administración de eventos (`Convención`, `Retiro Pastoral`, `Asamblea Nacional`, etc.) con múltiples días/sesiones (`ActividadSesion` con bandera oficial `is_activa` y control de quórum/comidas). El terminal web soporta 3 modos: **Carnet QR** (`jsqr`), **Biometría Facial** (`@vladmandic/face-api` con 14 archivos de modelos en `public/models` y enrolamiento en vivo) y **Uso Manual**, acreditación conjunta de cónyuges, reversión, reportes Excel/PDF (DomPDF) y operación **100% offline en navegador vía IndexedDB** (`MMM_Asistencia_Web_DB` + `sincronizarLote`) con concurrencia segura `lockForUpdate()`.

Seguridad y Brechas: Las operaciones CRUD administrativas carecen de `can:actividades.*`. Además, las 10 rutas operativas del escáner están duplicadas públicamente bajo `/admin/actividades/...` y `/actividades/...` con solo middleware `web` (sin `auth` ni PIN), permitiendo descargar el padrón offline completo, registrar/revertir asistencias, activar sesiones y sobreescribir fotos faciales (`enrolarRostro` sin validar extensión de imagen).

Módulo 7 — Aplicaciones Móviles / PWA (`asistencia-app` y `presbiterio-app`) y API v1

OPERATIVO REAL
KIOSKO SIN AUTH + SECRETOS ASYNCSTORAGE

Componentes: `AsistenciaApiController` , `PresbiterioApiController` , `ExpoPushNotificationService` , `asistencia-app/` , `presbiterio-app/` (compilada en `/presbitero`).

Alcance Operativo:
• **`ASISTENCIA CN2026` :** App móvil/PWA de puerta con escáner `expo-camera` , búsqueda manual, cola offline en `AsyncStorage` , sincronización según `sesion_activa` y padrón de fábrica de **899 pastores** (`padron-seed.json`).
• **`PRESBITERIO CN2026` :** Portal ejecutivo autenticado por Sanctum para Presbíteros (padrón de su zona, presentes/pendientes en vivo, alta automática de cónyuge y alertas vía **Expo Push API**) y Supervisor/Junta Nacional (ranking semafórico de las 41 zonas de Venezuela).

Seguridad y Brechas: Los 4 endpoints de liderazgo (`logout` , `push-settings` , `presbitero/dashboard` , `supervisor/dashboard`) exigen `auth:sanctum` y validación de rol, pero los tokens nunca caducan (`expiration => null`) y los 7 endpoints de `AsistenciaApiController` (`padron-offline` , `asistencias/registrar` , `activar-sesion`) son totalmente públicos. En el cliente `presbiterio-app` , `secureStore.ts` usa `AsyncStorage` en claro para guardar usuario + contraseña + PIN (**A2**) y el endpoint público `consultar-usuario` valida rol por cédula sin auth.

Módulo 8 — Estructura Organizacional (Empresas/Sucursales) y Geografía Territorial

OPERATIVO REAL
SIN CAN: EN RUTAS

Componentes: `EmpresaController` , `SucursalController` , `PaisController` , `EstadoController` , `MunicipioController` , `ParroquiaController` , `RegionalConfigurationService` .

Alcance Operativo: Gestión multiempresa y de sucursales (con logos, zona horaria, geolocalización y generación de `api_key` / `whatsapp_api_key`), configuración regional inyectada por `RegionalConfiguration` en cada petición web, y catálogos geográficos en 4 niveles (País → Estado → Municipio → Parroquia) con mapas interactivos.

Seguridad y Brechas: Todas las rutas territoriales y de empresas/sucursales solo validan `auth + verified` sin permisos `empresas.*` , `sucursales.*` , `paises.*` , `estados.*` , `municipios.*` o `parroquias.*` . En `Empresa` , `api_key` y `whatsapp_api_key` carecen de cifrado en BD y ocultamiento en `$hidden` .

Módulo 9 — Seguridad Administrativa, Usuarios, Roles (RBAC) y Recuperación

OPERATIVO REAL
RBAC SIN ENFORZAR + OTP DÉBIL

Componentes: `UserController` , `RoleController` , `SecurityController` , `ForceChangePasswordController` , `ForgotPasswordOtpController` , `FortifyServiceProvider` .

Alcance Operativo: CRUD de usuarios con asignación de zonas/distritos primarios y secundarios, estados (`activo` / `inactivo` / `suspendido`), matriz modular de roles/permisos (`PermissionOrganizer` + Spatie), envío automatizado de bienvenida y credenciales con clave temporal aleatoria (`random_int`) y bandera `must_change_password = true` , configuración de 2FA TOTP / Passkeys WebAuthn en `settings/security` y recuperación por código OTP de 6 dígitos vía WhatsApp.

Seguridad y Brechas: Rutas `/admin/usuarios/*` y `/admin/roles/*` sin middleware `can:users.*` / `roles.*` ; `Features::registration()` activo en Fortify (**C2**); recuperación OTP sin throttling ni bloqueo de intentos, usando `mt_rand()` y escribiendo el OTP en claro en `laravel.log` (**A4**); envío de claves temporales en claro por WhatsApp/Email CC y fallback legacy de contraseñas sin hash (**M2**); límite `max:12` en cambio forzado (**M3**) y error tipográfico `'inactivos'` en conteo de `UserController::index` .

Módulo 10 — Integraciones Externas y Comunicaciones

OPERATIVO REAL | WEBHOOK SIN FIRMA + PROPS EN CLARO

Componentes: `IntegrationController`, `WhatsAppService`, `WhatsAppWebhookController`, `MailNotificationService`, `JaakService`, `ControlAccesoService`.

Alcance Operativo: Panel centralizado por empresa para WhatsApp Business API (plantillas Spintax, anti-ban, blacklist, difusión masiva, reintentos y cron `whatsapp:heartbeat` cada 5 min), correo multi-proveedor (Google SMTP, Mailgun, Mailpit), mapas (Mapbox y Google Maps), KYC biométrico JAAK y Control de Acceso físico IVMS (con cron `asistencia:verificar-descansos` cada minuto).

Seguridad y Brechas: Las rutas administrativas sí aplican permisos granulares (`integrations.view/edit`, `whatsapp.*`, `jaak.*`) y `Empresa` cifra `jaak_api_key`, `google_smtp_password` y `mailgun_secret`. Sin embargo: (1) `IntegrationController::index` envía esos secretos desencriptados en props Inertia al navegador (**M1**); (2) `WhatsAppWebhookController` no valida firma HMAC ni secreto en webhooks exentos de CSRF (**A3**); y (3) `control_acceso_*_token` no está cifrado ni oculto en `Empresa`.

Módulo 11 — Monitoreo del Sistema, Respaldos BD y Auditoría

MIXTO (REAL + MÉTRICAS SIMULADAS)
RUTAS CRÍTICAS SIN CAN:

Componentes: `ActivityMonitoringController`, `DbMonitoringController`, `DatabaseExportService`, `DatabaseImportService`, `LogMonitoringController`, `QueueMonitoringController`, `ServerMonitoringController`, `SessionMonitoringController`, `TaskMonitoringController`.

Estado Operativo Real vs. Simulado:
• **100% Reales:** Auditoría Spatie ActivityLog (con eventos de login/logout, IP, User-Agent y diffs), listado real de tablas SQL, exportación `.sql/.sql.gz` e importación hasta 200 MB vía `DB::unprepared`, lectura/descarga/borrado de `laravel.log`, colas `jobs/failed_jobs`, sesiones activas (`sessions`) y cron `schedule:list`.
• **Simulados (`rand()`):** `ServerMonitoringController::getMetrics()` (CPU, RAM fija 16 GB, red, logs simulados) y `DbMonitoringController::getMetrics()` (QPS, conexiones, consultas ejemplo `leads/larareact`).

Seguridad y Brechas: Solo `activity-monitoring` (`can:monitoreo.activities`) y `registration-monitoring` (`can:monitoreo.view`) validan permisos. Los módulos de **Base de Datos (export/import), Logs, Colas, Sesiones, Servidor y Tareas Programadas** carecen de middleware `can:` y de verificación en controlador. Adicionalmente, `TaskMonitoringController::run` ejecuta cualquier comando `artisan *` enviado por el cliente sin lista blanca (**C1**).

04 Controles de Seguridad Existentes y Cobertura RBAC

El sistema combina mecanismos defensivos avanzados de Laravel 13 con brechas de aplicación desigual entre módulos. A continuación se documentan las fortalezas verificadas y la cobertura exacta de autorización sobre el total de 253 rutas.

Fortalezas Arquitectónicas Verificadas

ACTIVAS

- **Autenticación Multifactor y WebAuthn:** Fortify configurado con `twoFactorAuthentication(['confirm' => true, 'confirmPassword' => true])` y `Features::passkeys(['confirmPassword' => true])`, más limitadores de tasa dedicados (`throttle:login` 5/min, `throttle:two-factor` , `throttle:passkeys`).
- **Control de Estado de Cuenta y Cambio Forzado:** `FortifyServiceProvider` y `PresbiterioApiController` bloquean usuarios con estado `inactivo` o `suspendido` . `EnsurePasswordIsChanged` obliga a rotar la clave temporal antes de acceder al sistema.
- **Endurecimiento de Producción (`AppServiceProvider`):** `DB::prohibitDestructiveCommands(app()->isProduction())` impide comandos como `migrate:fresh` o `db:wipe` en producción, y `Password::defaults()` exige 12 caracteres, mayúsculas, minúsculas, números, símbolos y `uncompromised()` en producción.
- **Cifrado en Reposo en `Empresa` :** Secretos críticos de correo y KYC (`jaak_api_key` , `google_smtp_password` , `mailgun_secret`) usan cast `'encrypted'` con captura segura de `DecryptException` , ocultamiento en `$hidden` y exclusión en `logExcept()` .
- **Auditoría Forense Integral:** `AppServiceProvider` registra eventos Laravel `Login` , `Logout` y `Failed` en el canal `autenticacion` incluyendo IP y User-Agent; los modelos de dominio registran cambios sucios vía `LogsActivity` .
- **Concurrencia Atómica en Asistencia:** `AsistenciaController::registrar` y `AsistenciaApiController::registrarAsistencia` ejecutan `lockForUpdate()` dentro de `DB::transaction()` evitando duplicados en sincronizaciones simultáneas offline/online.
- **Re-autenticación en Operaciones Sensibles:** `settings/security` (`RequirePassword`), `DbMonitoringController` (`Hash::check` en exportación/importación BD) y `ExtensionController::verifySecurity` solicitan nuevamente la contraseña del usuario antes de proceder.

Cobertura de Autorización por Grupo de Rutas (253 Rutas Totales)

GRUPO DE RUTAS	CANT.	MIDDLEWARE / CONTROL OBSERVADO	ESTADO
Admin con permiso explícito (`integration` , `activity-monitoring` , `registration-monitoring`)	**45**	`web, auth, verified, can:*`	**PROTEGIDO**
Dashboard principal (`GET /dashboard`)	**1**	`web, auth, verified, can:dashboard.view`	**PROTEGIDO**
Admin de Carnets IA/PDF (verificación interna en controlador)	**12***	`web, auth, verified` + `autorizarAccesoCarnets()`	**CONTROL ROL INTERNO**
Resto de Admin (`usuarios` , `roles` , `pastores` , `extensiones` , `actividades` , `empresas` , `sucursales` , territorio, `db/logs/queues/server/sessions/tasks`)	**94***	`web, auth, verified` **SIN** `can:`	**SIN PERMISO RBAC**
Terminal Escáner bajo `/admin/actividades/*`	**10**	`web` (**sin** `auth` en `routes/web.php`)	**PÚBLICO**
API Liderazgo (`/api/v1/presbitero/*` , `supervisor/*` , `logout` , `push`)	**4**	`api, auth:sanctum` + chequeo de rol en supervisor	**TOKEN SANCTUM**
API Pública (Kiosko asistencia, auth login/consulta, webhook)	**10**	`api` o `web` (sin token ni firma)	**SIN AUTH / FIRMA**
Web Público, Fortify, OTP, Registro Público, Settings, DevTools	**77**	`web` / `guest` / `auth` / `EnsurePublicRegistrationIsActive`	**MIXTO**

* Del total de 106 rutas `/admin/*` sin middleware `can:` reportadas por el inspector de rutas, 12 corresponden a endpoints de carnets/fotos en `PastorCarnetController` protegidos internamente mediante `autorizarAccesoCarnets()` , y las 94 restantes carecen tanto de `can:` en ruta como de validación de rol en controlador.

Matriz Comparativa de Cifrado, Almacenamiento y Sesiones

CAPA / DATO SENSIBLE	MECANISMO OBSERVADO	EVALUACIÓN
Contraseñas de Usuarios Web/API	Bcrypt (`BCRYPT_ROUNDS=12`), cast `hashed` en `User` , pero persiste fallback de comparación en claro en `FortifyServiceProvider` y `PresbiterioApiController` .	**MEJORAR (RETIRAR LEGACY)**
Sesiones Web (`config/session.php`)	Driver `database` , TTL 120 min, `http_only = true` , `same_site = 'lax'` , serialización `json` ; actualmente `SESSION_SECURE = null` y `encrypt = false` en local.	**ENDURECER EN PROD**
Credenciales de Integración (`Empresa`)	`jaak_api_key` , `google_smtp_password` y `mailgun_secret` cifrados y en `$hidden` ; `api_key` , `whatsapp_api_key` y `control_acceso_*` sin cifrar ni ocultar; props enviadas en claro a UI.	**PARCIAL**
Tokens y Credenciales en App Móvil (`presbiterio-app`)	Sanctum con `expiration => null` ; cliente guarda token, PIN y contraseña literal en `AsyncStorage` (`localStorage` en navegador).	**VULNERABLE (A2)**
Documentos de Identidad (`foto_cedula`)	`PastorRegistroPublicoController::store` guarda en disco `public` (`pastores_cedulas`) accesible web directamente sin pasar por `descargarCedula` .	**EXPOSICIÓN PÚBLICA**

05 Registro Técnico de Hallazgos y Remediaciones

Detalle técnico verificado para cada vulnerabilidad o brecha de configuración identificada, incluyendo evidencia directa en el código fuente y acciones concretas de corrección.

C1 — Control de Acceso Roto (BOLA/RBAC) en 94 Rutas Admin + Ejecución Arbitraria de Artisan y Acceso a BD

CRÍTICO

Evidencia: Escaneo de controladores y `Route::getRoutes()`: `routes/modules/{usuarios,roles,pastores,extensiones,actividades,empresas,sucursales,paises,estados,municipios,parroquias,db-monitoring,log-monitoring,queue-monitoring,server-monitoring,session-monitoring,task-monitoring}.php`.

IMPACTO TÉCNICO VERIFICADO

Cualquier usuario autenticado con email verificado —sin importar su rol (ej. `Presbitero` o cuenta auto-registrada vía C2)— puede acceder a `/admin/monitoring/database/export` e `import` (solo validan la contraseña *propia* del usuario atacante, ejecutando `DB::unprepared()` sobre SQL hasta 200 MB), descargar/vaciar `laravel.log`, revocar sesiones de administradores, editar roles/permisos en `/admin/roles` y disparar cualquier comando `artisan *` en `TaskMonitoringController::run()` (`Artisan::call(substr($command, 8))`).

REMEDIACIÓN RECOMENDADA

Aplicar middleware `can:<modulo>.<accion>` o `role:Super Administrador` en los 17 archivos de rutas faltantes; implementar una lista blanca estricta de comandos permitidos en `TaskMonitoringController::run` ; exigir `can:monitoreo.backup` / `monitoreo.import` + rol `Super Administrador` en respaldo BD; y agregar `FormRequest` / `Policy` en todos los controladores REST.

C2 — Auto-Registro Público de Usuarios Fortify Habilitado (`/register`)

CRÍTICO

Evidencia: `config/fortify.php` (línea 164: `Features::registration()`), rutas activas `GET/POST register` , y `app/Actions/Fortify/CreateNewUser.php`.

IMPACTO TÉCNICO VERIFICADO

Cualquier visitante externo puede crear un usuario en la tabla `users` desde `/register` . Aunque no asigna rol ni `empresa_id` , un usuario recién registrado adquiere sesión autenticada y, combinado con la ausencia de `can:` en el panel administrativo (C1) —si verifica su correo o ingresa a rutas sin `verified` —, puede interactuar con módulos internos y evadir el scope territorial por falta de zona/empresa asignada.

REMEDIACIÓN RECOMENDADA

Comentar o deshabilitar `Features::registration()` en `config/fortify.php` si el alta de operadores administrativos es exclusivamente interna por `UserController` . Además, hacer que `User` implemente `MustVerifyEmail` o denegar el acceso administrativo a usuarios sin rol asignado.

C3 — Endpoints de Asistencia Web y API v1 Sin Autenticación + Padrón Nacional Expuesto en Kiosko/Bundle

CRÍTICO

Evidencia: `routes/web.php` (líneas 43-76: rutas duplicadas en `/admin/actividades/*` y `/actividades/*` solo con middleware `web`), `routes/api.php` (líneas 16-24: `/api/v1/pastores/padron-offline` , `/pastores/buscar` , `/asistencias/registrar` , `/activar-sesion`) y `asistencia-app/src/data/padron-seed.json` (899 pastores, 520.886 bytes).

IMPACTO TÉCNICO VERIFICADO

Sin autenticación ni token de terminal autorizado, cualquier tercero en Internet puede: (1) descargar el padrón completo nacional (`padron-offline`), (2) consultar pastores por nombre/cédula, (3) marcar asistencias masivas o borrarlas vía `DELETE .../revertir` , (4) cambiar la sesión activa oficial del evento (`activar-sesion`), y (5) extraer 899 registros del archivo estático `padron-seed.json` incluido en el paquete cliente de la app de asistencia.

REMEDIACIÓN RECOMENDADA

Proteger el escáner y las rutas API de puerta mediante autenticación Sanctum con rol dedicado (ej. `Operador Kiosko`) o mediante tokens firmados por dispositivo/evento con expiración por jornada; restringir `activar-sesion` y `DELETE /revertir` exclusivamente a usuarios administradores autenticados; y retirar `padron-seed.json` estático del código fuente público.

C4 — Exposición de PII/Salud, Toma de Fichas en `/registro`, Subida sin MIME y Cédulas en Disco Público

CRÍTICO

Evidencia: `PastorRegistroPublicoController.php` (`index` líneas 45-78, `verificarCedula` líneas 282-346, `store` líneas 382-719 y 496-595), `AsistenciaController::enrolarRostro` (líneas 353-378).

IMPACTO TÉCNICO VERIFICADO

Con el registro público abierto: (1) `GET /registro` filtra en el HTML inicial el listado completo de pastores activos; (2) `GET /registro/verificar-cedula/{cedula}` devuelve datos médicos (grupo sanguíneo, enfermedades, alergias, medicamentos) y de contacto/domicilio sin autenticación; (3) `POST /registro` actualiza fichas existentes por `pastor_id` / `documento` sin verificar identidad (toma de ficha); (4) `foto` y `foto_cedula` carecen de validación `image|mimes` (y `enrolarRostro` acepta cualquier `\w+` como extensión); y (5) `foto_cedula` se almacena en `storage/app/public/pastores_cedulas` y `public/pastores_cedulas`.

REMEDIACIÓN RECOMENDADA

Requerir desafío de verificación (OTP por WhatsApp/email registrado o token temporal por enlace institucional) antes de pre-rellenar o actualizar fichas existentes; no serializar la lista nacional en `index()`; aplicar validación estricta `['nullable', 'image', 'mimes:jpg,jpeg,png,webp', 'max:5120']` y lista blanca de extensiones base64; y mover `pastores_cedulas` al disco privado `local` servido solo vía `descargarCedula` autorizado.

A1 — Microservicio Python Studio AI Sin Autenticación, CORS Wildcard y Rutas Arbitrarias en Disco

ALTO

Evidencia: `app/Scripts/ai_server.py` (líneas 48-54, 101-186, 215) y `app/Services/PhotoAiEnhancerService.php` (líneas 75-84).

IMPACTO TÉCNICO VERIFICADO

El servidor Uvicorn enlaza en `0.0.0.0:8001` (expuesto en red) con `allow_origins=["*"]` y `allow_credentials=True`, sin cabecera secreta compartida. Endpoints como `/enhance-file`, `/analyze` y `/remove-background` reciben cadenas arbitrarias `input_path` y `output_path` y operan directamente sobre el sistema de archivos del servidor sin validación de directorio raíz permitido.

REMEDIACIÓN RECOMENDADA

Enlazar localmente en `127.0.0.1:8001` cuando sea co-alojado (o firewall IP privado + TLS si es remoto); exigir un encabezado secreto obligatorio (`X-Studio-AI-Key` con comparación en tiempo constante); restringir CORS; y sanear `input_path` / `output_path` mediante `os.path.realpath()` asegurando pertenencia estricta a la carpeta de fotos permitida.

A2 — Contraseñas y PIN Guardados en Texto Plano en `AsyncStorage` y Biometría Solo Cliente (`presbiterio-app`)

ALTO

Evidencia: `presbiterio-app/src/services/secureStore.ts` (usa `AsyncStorage`), `src/context/AuthContext.tsx` (líneas 165-168: guarda `{ usuario, pass, name }`; 180-240), y `src/services/biometricService.ts` (líneas 144-250).

IMPACTO TÉCNICO VERIFICADO

Pese al nombre `secureStore.ts`, las credenciales completas del Presbítero/Supervisor (`pass` en texto claro), el PIN de acceso rápido y el token Sanctum se almacenan en `AsyncStorage` (equivalente a `localStorage` en navegador cuando se usa la PWA `/presbitero`). Asimismo, WebAuthn en web valida un desafío generado por el propio cliente sin verificación criptográfica de firma en Laravel.

REMEDIACIÓN RECOMENDADA

Eliminar por completo el guardado de contraseñas (`pass`) en el cliente; emplear únicamente renovación segura de tokens Sanctum de corta duración en `expo-secure-store` nativo (Keychain/Keystore) y cookies `HttpOnly` o re-autenticación WebAuthn verificada por el servidor (`@laravel/passkeys`) en la versión PWA web.

A3 — Webhook de WhatsApp Exento de CSRF Sin Verificación de Firma o Secreto Compartido

ALTO

Evidencia: `bootstrap/app.php` (líneas 69-70: `validateCsrfTokens(except: ['webhooks/whatsapp', 'api/webhooks/whatsapp'])`) y `app/Http/Controllers/WhatsAppWebhookController.php` (líneas 13-121).

IMPACTO TÉCNICO VERIFICADO

El método `handleIncoming` confía ciegamente en los campos JSON `event`, `instanceName` y `data` sin validar token Bearer ni firma HMAC del gateway emisor. Un atacante externo puede enviar peticiones POST simulando `event=contact.opt_out` para desactivar notificaciones WhatsApp a usuarios legítimos, falsificar mensajes entrantes en `whatsapp_messages` o alterar estados de entrega (`delivered` / `read`).

REMEDIACIÓN RECOMENDADA

Implementar verificación obligatoria de firma HMAC-SHA256 (o token secreto `X-Webhook-Secret` almacenado cifrado en configuración) comparando con `hash_equals()` antes de procesar cualquier evento en `WhatsAppWebhookController`.

A4 — Recuperación de Contraseña vía OTP WhatsApp Sin Límite de Tasa, Uso de `mt_rand()` y Log en Claro

ALTO

Evidencia: `routes/web.php` (líneas 79-84: rutas `forgot-password/*` sin `throttle`) y `app/Http/Controllers/Auth/ForgotPasswordOtpController.php` (líneas 58, 76, 105-146, 156).

IMPACTO TÉCNICO VERIFICADO

(1) `sendOtp` genera el código con `mt_rand()` (no CSPRNG) y lo imprime en texto claro en `storage/logs/laravel.log` (línea 76), exponiendo códigos de recuperación en el lector de logs; (2) `verifyOtp` no tiene contador de intentos fallidos ni middleware `throttle:`, permitiendo ataques automatizados de fuerza bruta contra los 6 dígitos en la ventana de 10 minutos; (3) `resetPassword` solo valida `min:8` ignorando `Password::defaults()`.

REMEDIACIÓN RECOMENDADA

Reemplazar `mt_rand()` por `random_int(100000, 999999)`; eliminar la impresión del OTP en logs; almacenar el código hasheado en caché con contador máximo de 5 intentos erróneos antes de invalidarlo; añadir `throttle:5,1` en rutas OTP; y aplicar `Password::defaults()` en el restablecimiento.

Hallazgos de Severidad Media y Baja

M1 — Exposición de Secretos de Integración en Props Inertia, Campos Sin Ocultar en `Empresa` y Hashes en Logs

MEDIO

Evidencia: `IntegrationController::index` (líneas 52, 57, 71-72), `HandleInertiaRequests.php` (líneas 49-52), `Empresa.php` (líneas 24-38, 111-134) y `User.php` (líneas 48, 55-62).

DETALLE TÉCNICO

`IntegrationController::index` entrega en claro al DOM de Inertia `google_smtp_password`, `mailgun_secret`, `control_acceso_app_token` y `control_acceso_user_token`. En `Empresa`, `api_key`, `whatsapp_api_key` y `control_acceso_*_token` no usan cast `encrypted` ni están en `$hidden` / `logExcept()`. En `User`, `password` figura en `#[Fillable]` y `getActivitylogOptions()` usa `->logFillable()` sin `->logExcept(['password'])`, guardando cambios de hash en `activity_log.properties`.

REMEDIACIÓN

Enmascarar secretos en props de Inertia (devolver bandera booleana `has_secret` en lugar del valor real y actualizar solo si se envía nuevo valor); agregar `encrypted` y `$hidden` / `logExcept()` a todas las API keys en `Empresa`; y añadir `->logExcept(['password'])` en `User::getActivitylogOptions()`.

M2 — Contraseñas Temporales por Canales Externos con CC Fijo, Fallback Legacy en Claro y Clave de Prueba WhatsApp

MEDIO

Evidencia: `UserController.php` (líneas 275-529: CC a `oficina@mmmvenezuela.org`), `FortifyServiceProvider.php` (línea 60), `PresbiterioApiController.php` (línea 32) y `WhatsAppService.php` (línea 88: `'test-api-key-vargas-centro'`).

DETALLE TÉCNICO

Los correos de bienvenida/credenciales envían la contraseña temporal en claro con copia automática a `oficina@mmmvenezuela.org` y también por WhatsApp. Aunque se exige `must_change_password = true`, las claves iniciales quedan persistidas en historial de correo/chat. Asimismo, persiste la comparación literal en claro `$password === $user->password` en web y API, y una clave fallback predefinida en `WhatsAppService`.

REMEDIACIÓN

Sustituir el envío de contraseñas en claro por enlaces firmados temporales de activación (`Password::broker()`) y retirar la copia CC de credenciales; ejecutar una migración única de re-hash y eliminar el fallback de texto plano en `FortifyServiceProvider` y `PresbiterioApiController`; y fallar explícitamente si falta `whatsapp_api_key`.

M3 — Contradicción de Política de Contraseñas (`max:12` vs `min:12`) y Tokens Sanctum Sin Caducidad

MEDIO

Evidencia: `ForceChangePasswordController.php` (líneas 78-82: `min:8|max:12`), `UserController::sendCredentials` (líneas 405-407), `AppServiceProvider.php` (líneas 89-95: `min(12)` en producción) y `config/sanctum.php` (línea 53: `'expiration' => null`).

DETALLE TÉCNICO

Mientras `AppServiceProvider` impone al menos 12 caracteres en producción, el formulario de cambio obligatorio y `sendCredentials` prohíben superar los 12 caracteres (`max:12`), restringiendo contraseñas largas y fraccionando la política. Paralelamente, los tokens móviles Sanctum nunca caducan salvo logout explícito.

REMEDIACIÓN

Estandarizar todos los validadores de contraseña sobre `Password::defaults()` (elevando o retirando `max:12` hasta al menos `max:128`); definir `'expiration' => 43200` (ej. 30 días) o rotación periódica en `config/sanctum.php` y emitir tokens con `abilities` específicas.

M4 — Ausencia de Cabeceras de Seguridad HTTP, CORS Abierto (`*`) y `APP_KEY` en `.env.example`

MEDIO

Evidencia: `app/Http/Middleware/`, `public/.htaccess` (líneas 1-29), `config/cors.php` (línea 22: `'allowed_origins' => ['*']`) y `.env.example` (línea 3).

DETALLE TÉCNICO

Ni Apache ni Laravel inyectan cabeceras defensivas (`Content-Security-Policy`, `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Strict-Transport-Security`). Además, `cors.php` autoriza origen wildcard en `api/*` y `.env.example` incluye una `APP_KEY` pre-poblada en el repositorio.

REMEDIACIÓN

Añadir un middleware global de cabeceras de seguridad HTTP; restringir `allowed_origins` en `config/cors.php` a los dominios oficiales de la institución y de las apps móviles; limpiar `APP_KEY=` en `.env.example` y rotar la llave en producción.

B1 — Enumeración QR Pública

BAJO

`GET /validar-credencial/{codigo}` acepta `id` secuencial o cédula sin rate limiting, permitiendo enumerar pastores. **Fix:** Usar código/token HMAC opaco y aplicar `throttle:30,1`.

B2 — Monitoreo Simulado

BAJO

`ServerMonitoringController` y `DbMonitoringController` generan métricas en vivo con `rand()` y SQL ficticio (`leads`). **Fix:** Conectar a `sys_getloadavg()`, `/proc/meminfo` y `SHOW GLOBAL STATUS` o etiquetar claramente como demo.

B3 — Typo, Legado y Entorno

BAJO

`UserController::index` consulta `status='inactivos'` (typo por `inactivo`). Persisten permisos/comandos LFT residuales y entorno activo con `APP_DEBUG=true`, `SESSION_SECURE=null`.

06 Plan de Acción Priorizado (Hoja de Ruta de Remediación)

Secuencia recomendada de ejecución por fases para cerrar las brechas críticas antes del despliegue público y estabilizar los controles operativos restantes.

FASE / VENTANA	PRIORIDAD	ACCIONES CONCRETAS DE ENDURECIMIENTO	MÓDULOS / ARCHIVOS CLAVE
Fase 1 Inmediata (0–48 h)	**BLOQUEANTE**	1) Proteger las 94 rutas administrativas faltantes con middleware `can:` y restringir `DbMonitoringController` + `TaskMonitoringController` (lista blanca de comandos Artisan). 2) Deshabilitar `Features::registration()` (`/register`). 3) Cerrar acceso anónimo al padrón offline y al escáner web/API exigiendo sesión/token de operador autorizado, y retirar `padron-seed.json` . 4) Mientras no se implemente verificación OTP en `/registro` , mantener cerrado el registro público y añadir validación `image|mimes` + almacenamiento privado para `pastores_cedulas` .	`routes/modules/*.php` , `routes/web.php` , `routes/api.php` , `config/fortify.php` , `PastorRegistroPublicoController` , `AsistenciaController`
Fase 2 Corto Plazo (1–2 sem)	**ALTA**	1) Asegurar `ai_server.py` en `127.0.0.1:8001` con API Key compartida y canonicalización de rutas. 2) Eliminar guardado de contraseñas en claro en `presbiterio-app` , migrar a `expo-secure-store` en móvil y passkeys verificadas en servidor para PWA web. 3) Verificar firma HMAC/secreto en `WhatsAppWebhookController` . 4) Endurecer flujo OTP WhatsApp (`random_int` , sin log en claro, bloqueo tras 5 intentos, `throttle:`).	`app/Scripts/ai_server.py` , `presbiterio-app/src/` , `WhatsAppWebhookController` , `ForgotPasswordOtpController`
Fase 3 Mediano Plazo (2–4 sem)	**MEDIA**	1) Enmascarar secretos en props Inertia de integraciones, cifrar `api_key` / `whatsapp_api_key` / `control_acceso_*` y excluir `password` de `User` ActivityLog. 2) Reemplazar envío de claves temporales en claro/CC por enlaces firmados de primer ingreso y retirar comparación legacy de contraseñas en claro. 3) Unificar `Password::defaults()` (retirar `max:12`), configurar caducidad en `sanctum.php` , agregar cabeceras HTTP de seguridad y restringir CORS.	`IntegrationController` , `Empresa.php` , `User.php` , `UserController` , `ForceChangePasswordController` , `cors.php` , `sanctum.php`
Fase 4 Pre-Producción	**ESTABILIZACIÓN**	1) Configurar `APP_ENV=production` , `APP_DEBUG=false` , `SESSION_SECURE_COOKIE=true` , `SESSION_ENCRYPT=true` y rotar `APP_KEY` (limpiar `.env.example`). 2) Reemplazar métricas simuladas de Servidor/BD por lectura real del sistema o desactivar la vista sintética, corregir typo `'inactivos'` en `UserController` , usar token firmado en verificación QR y depurar comandos/permisos LFT obsoletos.	`.env` , `.env.example` , `ServerMonitoringController` , `DbMonitoringController` , `PastorCarnetController` , `PermissionSeeder`

Conclusión de Auditoría: El sistema **SAPRCOE — MMM Venezuela** cuenta con una base funcional robusta y moderna (Laravel 13, Inertia/React 19, biometría ONNX y operación offline multi-dispositivo). Al ejecutar de forma prioritaria las Fases 1 y 2 sobre control de acceso RBAC, endurecimiento de endpoints públicos y almacenamiento móvil seguro, la plataforma alcanzará un nivel de madurez sólido y apto para operación institucional nacional continua.